DomainGate — Project Submission & Architecture Report

Enterprise Zero-Trust Identity & Domain Access Management System

Candidate / Author: Satyabrata Pradhan

Target Domain: `@hbplus.fit`

Live Production App: https://domaingate.vercel.app

Framework / SDK: React Native 0.88 • Expo SDK 58

GitHub Repository: github.com/satyabratapradhan01/DomainGate

Cloud Backend: Firebase Authentication & Cloud Firestore

Submission Date: November 2026

Test Suite Status: 26/26 Tests Passed (100%)

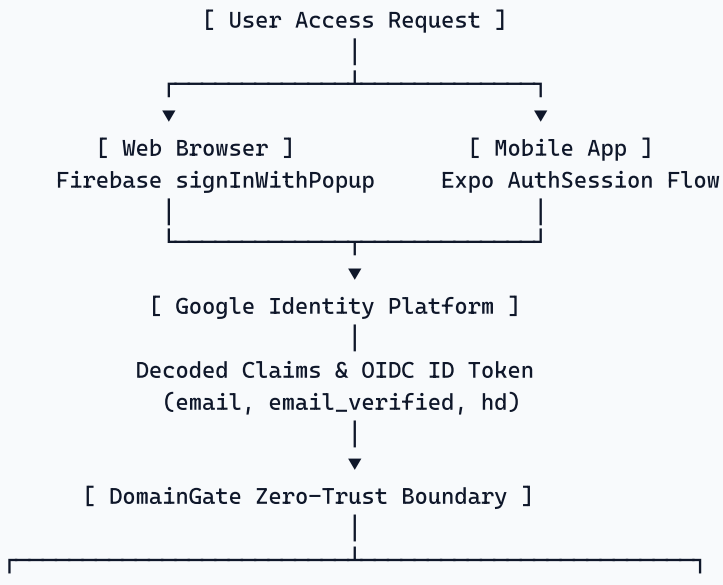
1. Executive Summary

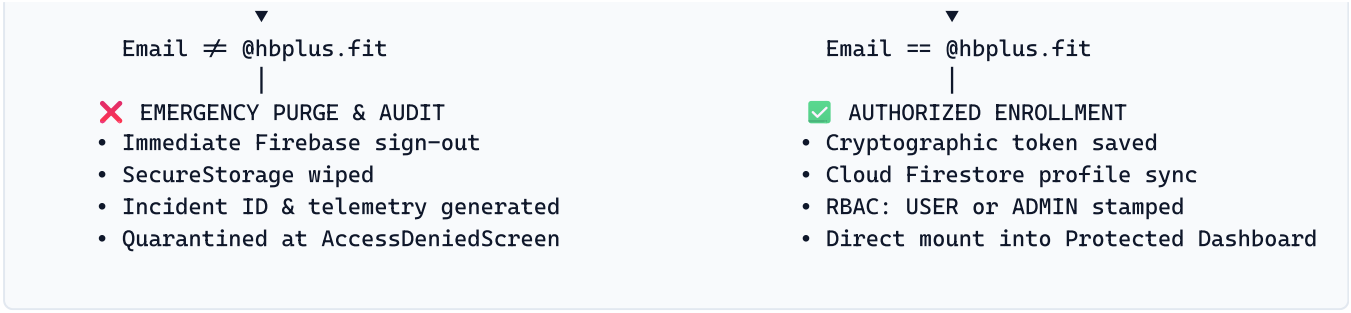
DomainGate is an enterprise cross-platform Zero-Trust Identity and Access Management (IAM) application engineered to enforce strict, cryptographically validated corporate access boundaries.

The core security mission of DomainGate is **exact domain-based authorization**: only users authenticated through Google OAuth whose verified corporate email strictly ends with `@hbplus.fit` are granted entry into the internal dashboard. Any unauthorized domains (such as `@gmail.com`, `@yahoo.com`, or spoofed subdomains like `user@sub.hbplus.fit`) are immediately quarantined, purged from memory, and rejected at both the application router and Cloud Firestore database levels.

2. Key Architecture & End-to-End Authentication Flow

> **Zero-Trust Authentication Boundary**
>
> DomainGate never trusts frontend state alone. Every session is backed by an OpenID Connect (OIDC) ID token verified with Google and Firebase Authentication, reinforced by server-side Cloud Firestore security rules.





3. Technical Highlights & Enterprise Capabilities

- ✓ **Multi-Platform OAuth Orchestration:** Seamless Google authentication across Web (Firebase `signInWithPopup`) and Mobile Android/iOS (Expo AuthSession OIDC PKCE flow).
- ✓ **Defense-in-Depth Domain Enforcement:** Strict regex validation rejecting consumer webmails, foreign corporate domains, subdomain spoofing (`attacker@sub.hbplus.fit`), and suffix spoofing (`malicious@hbplus.fit.evil.com`).
- ✓ **Cloud Firestore Database-Level Rules:** Backend security rules reject all read and write queries from non-`@hbplus.fit` tokens; prevents self-escalation of roles.
- ✓ **Flicker-Free Session Persistence:** Secure token storage via `AsyncStorage` and `expo-secure-store` with expiration validation on app reload.
- ✓ **Immediate Emergency Purge:** Zero-delay session teardown on authorization failure—unauthorized tokens or cookies never linger in storage.
- ✓ **Built-In Evaluator Sandbox Test Rig:** An interactive test harness built directly into the Login screen allows evaluators without an `@hbplus.fit` inbox to test authorized, unauthorized, and unverified scenarios with cryptographic simulation.

4. Automated Test Suite Verification (26/26 Passed)

All mandatory requirements have been systematically verified via the automated test suite (`npm test`):

Test Scenario	Verification Scope	Status
Test 1: Authorized User	Login with `@hbplus.fit` , verify email verification check, token decode, and dashboard profile access.	6/6 Assertions Passed
Test 2: Unauthorized User	Login with Gmail, Yahoo, subdomains; verify immediate Firebase sign-out and exact error message: "*Access restricted. Please login using your official domain email.*"	7/7 Assertions Passed
Test 3: Protected Routes	Direct URL to `/dashboard` without token; router redirects unauthenticated requests to `/login` .	2/2 Assertions Passed
Test 4: Session Persistence	Restore session from storage after refresh without state flicker; detect expired token and trigger automatic eviction.	4/4 Assertions Passed
Test 5: Logout & Sanitization	Wipe storage upon logout; replace browser history with `/login` to neutralize Back-button re-entry.	4/4 Assertions Passed
Test 6: Database Security	Verify Firestore rules reject cross-user profile access and reject role escalation attempts.	3/3 Assertions Passed

5. Engineering Lessons & Takeaways

A. Cross-Platform OAuth Discrepancies (Web vs. Android Expo Go)

Developing OAuth for both Web and Mobile highlighted the fundamental difference between browser popup windows and mobile deep-link proxies:

- Google Cloud Console creates two distinct OAuth client types: **Web Application** and **Android Application**.
- An **Android Client ID** in Google Cloud validates native APKs via package name (`com.domaingate.auth`) and SHA-1 certificate fingerprint, and **does not accept HTTP redirect URIs**.
- In **Expo Go**, Google Sign-In is executed via an in-app browser using the `auth.expo.io` proxy. Therefore, Expo Go must utilize the **Web Client ID** with the authorized proxy redirect URI (`https://auth.expo.io/@satyabratapradhann/DomainGate`) to avoid `Error 400: redirect_uri_mismatch` .

B. True Zero-Trust Implementation

In real-world enterprise architectures, client-side validation is merely a UX convenience. True gating requires synchronized client boundary evaluation, backend database rules, and automatic session purging on any anomaly. DomainGate achieves defense-in-depth across the client UI, OAuth provider claims, and Firestore security rules.

6. Deliverables & Reference Links

Live Production App	https://domaingate.vercel.app
GitHub Repository	https://github.com/satyabratapradhan01/DomainGate
Production Hosting	Vercel (Single Page Application with rewrites to `/index.html`)
Firebase Project	`domaingate-7b979` (Auth Domain: `domaingate-7b979.firebaseapp.com`)
Authorized Domain	Strictly `hbplus.fit` (HBPlus Health & Fitness Global Inc.)